

Exercise 2: Regression Trees & Random Forests

Implementation and Experiments

Machine Learning WS2025
Jakob Olsacher, Tobias Ponesch, Lisa Winkler

TU Wien - Faculty of Informatics

December 14, 2025

Outline

- 1 Data Sets
- 2 Implementation Details
- 3 Experimental Results
- 4 Conclusion
- 5 References

Dataset 1: Pollen

- **Source:** OpenML [1]
- **Characteristics:**
 - Domain: Density of Pollen
 - Samples (N): 3848 (**small**)
 - Features (D): 6 (**low**)
- **Target Variable:** DENSITY

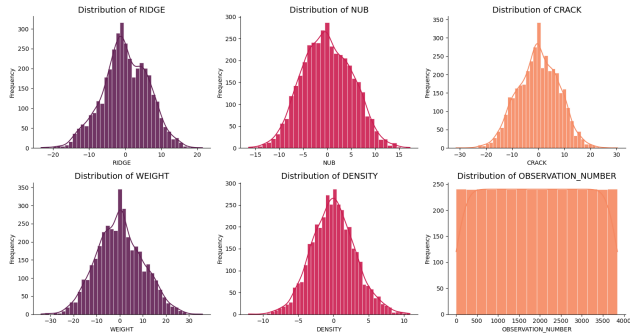


Figure 1: Distribution of Features in Pollen Dataset

Dataset 2: Spotify Dataset for ML Practice

- **Source:** Kaggle [2]
- **Characteristics:**
 - Domain: Spotify song popularity
 - Samples (N): $\approx 114,000$
 - Features (D): 158: 126 binary (incl. One-Hot), 31 numerical
- **Target variable:** Popularity Regression
- **Key difference vs. Dataset 1:** More samples and much higher dimensionality
- **Note:** Due to horrendous fit times, the dataset was subsampled to use only 30% of the original data, using 10 bins and then doing a 80/20 train-test split.

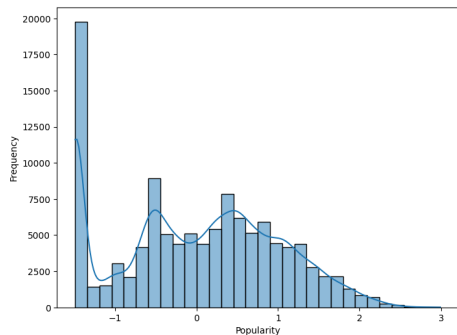


Figure 2: Distribution of the target variable: Popularity

Dataset 3: Taiwan Air Quality

- **Source:** Kaggle[3]
- **Characteristics:**
 - Domain: Air quality
 - Samples (N): approximately 5 million
 - Features (D): 15 numerical features
- **Target variable:** Air quality index
- **Key difference:** Significantly larger sample size compared to the other datasets
- **Note:** The dataset was subsampled due to computational constraints

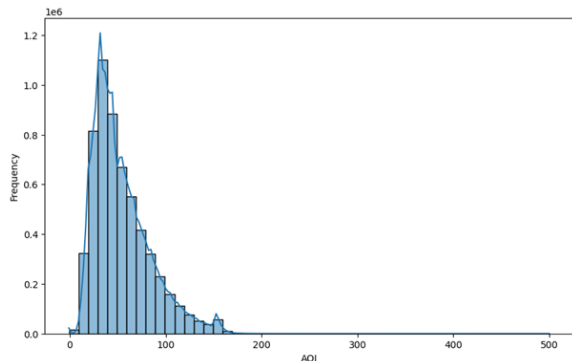


Figure 3: Distribution of AQI

Custom Implementation vs. Scikit-Learn

Feature	Our Implementation	Scikit-Learn
Language	Python + NumPy	Cython (C-Extension)
Speed	Slower (Interpreter overhead)	Highly Optimized
Parallelism	Joblib (Thread-based)	Joblib / OpenMP
Split Criteria	RSS (MSE)	MSE, MAE, Poisson, etc.
Missing Values	Not handled	Can handle (depends on version)

Note: While our logic for finding the split is mathematically identical (MSE minimization), sklearn's low-level optimization allows it to scale better to massive datasets.

Regression Tree

- `TreeNode-`, `Splitter-` and `RegressionTree-` Classes.
- Recursive splitting to...
- minimize Residual Sum of Squares (RSS) until...
- at least one stopping criteria is reached.
- Parallel Execution (threading Backend)

Random Forest

- Ensemble of Regression Trees, using the custom Regression Tree.
- Bootstrap Aggregation (Bagging).
- Allows for `max_features` to be defined, which allows random feature selection at each node split.
- Parallel Execution (1oky Backend: processes).

Regression Tree: The Node Structure

TreeNode class design:

- Stores split information: `feature`, `threshold`.
- Stores structure: `left`, `right` children.
- Leaf Nodes: Store the value (mean of target).

Stopping Criteria implemented:

- 1 `max_depth` reached.
- 2 `min_samples_split` not met.
- 3 `min_samples_leaf` violation.
- 4 Target variance is zero (pure node).

Regression Tree: Efficient Splitting (1/2)

We implemented an efficient split search strategy.

Algorithm Step 1: Sorting

For each feature, samples are sorted by their feature values. Sorting ensures that valid split candidates are evaluated in order and enables efficient, sequential RSS computation.

```
1 # regressionTree.py snippet
2 def _calculate_best_split_for_feature(self, df, feature):
3     X_col = df[feature].values
4     y_col = df[self.target].values
5
6     # sort by feature
7     order = np.argsort(X_col)
8     x_sorted = X_col[order]
9     y_sorted = y_col[order]
10
```

Regression Tree: Efficient Splitting (2/2)

Algorithm Step 2: Vectorized Prefix Sums

All candidate splits are evaluated simultaneously using vectorized NumPy operations. Prefix sums allow computing the RSS for every split in a single pass without explicit loops.

```
1 # Vectorized RSS computation using prefix sums
2 cum_y    = y_sorted.cumsum()
3 cum_y2   = (y_sorted ** 2).cumsum()
4
5 # RSS for all split positions computed at once
6 rss_left  = cum_y2[:-1] - (sum_left ** 2) / n_left
7 rss_right = ...
8 total_rss = rss_left + rss_right
9
```

→ Significant speedup through vectorization instead of iterative loops.

Regression Tree: Parallelization

- Evaluating a split requires testing all candidate features.
- Feature-wise split evaluation is parallelized using `joblib.Parallel`.
- The Regression Tree parallelizes using the `threading` backend, which is threadwise parallelization (shared memory).

Parallelization strategy:

- For large nodes ($N > 1000$), multiple threads are used to evaluate features concurrently.
- For small nodes, parallelization is disabled since the overhead exceeds the performance gain.

Random Forest: Core Components

Initialization

- Number of trees: `n_estimators`
- Feature selection at split: `max_features` → `'sqrt'`, `'log2'`, float ratio, or `None` (all features)
- Tree stopping criteria: `max_depth`, `min_samples_split`, `min_samples_leaf`

Training (`fit`):

- Uses `joblib` to train trees in parallel (loky backend – process wise parallelization, not shared memory)
- Each tree gets a random seed for reproducibility.

Random Forest: Bootstrapping

Implementation of Bagging (Bootstrap Aggregating):

```
1 def _get_bootstrap_samples(self, df, random_state):
2     bootstrap_df = df.sample(
3         frac=1,
4         replace=True,      # Sampling with replacement
5         random_state=random_state
6     )
7     # Remaining indices form the Out-Of-Bag (OOB) set
8     oob_indices = df.index.difference(bootstrap_indices)
9
```

Random Forest: Feature Subsampling

To decorrelate trees, we don't look at all features for every split.

Implementation Strategy:

- The `RandomForest` class calculates the subset size k (e.g., $\sqrt{D}$).
- Passed down to the `RegressionTree`.
- The `Splitter` inside the tree randomly selects k features before finding the best split.

$$k = \begin{cases} \sqrt{D} & \text{if 'sqrt'} \\ \log_2(D) & \text{if 'log2'} \\ n & \text{if type(n) == int} \\ n \times D & \text{if type(n) == float} \\ D & \text{if None} \end{cases}$$

Random Forest: Prediction

Aggregation of results via Mean.

```
1 def predict(self, X):  
2     # Collect predictions from all trees in parallel  
3     tree_predictions = self._generate_tree_predictions(X)  
4  
5     # Concatenate results  
6     preds_df = pd.concat(tree_predictions, axis=1)  
7  
8     # Return mean across all trees (columns)  
9     return preds_df.mean(axis=1)  
10
```

- **Numerical Stability**

- Issue: Large target values caused float overflow in RSS calculation.
- → Solution: Shift y by subtracting the mean/first element before squaring.

- **Performance Overhead**

- Issue: Parallelization has overhead.
- → Solution: Use parallelization only for nodes with $N > 1000$.

- **Identical Feature Values**

- Issue: Splitting without information gain
- → Solution: Add logic to prevent splitting if a feature is constant ($\text{nunique()} \leq 1$).

Experiments & Performance Metrics

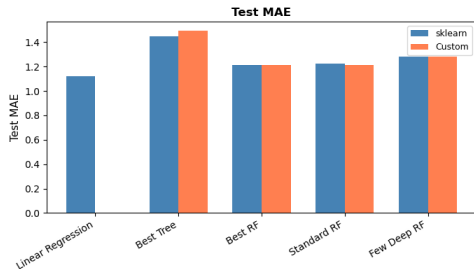
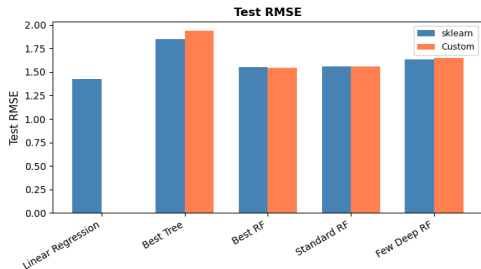
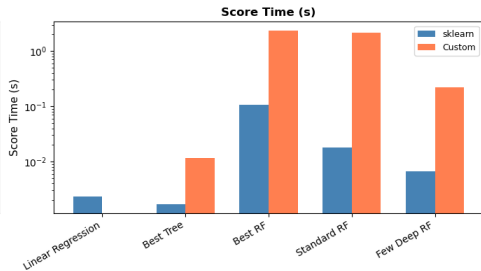
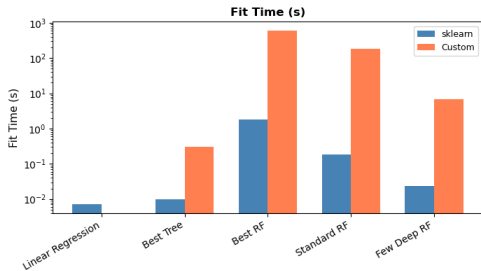
- Cross Validation
- Holdout Validation
- Overfitting
- Slowdown

$$\text{RMSE}(y, \hat{y}) = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} (y_i - \hat{y}_i)^2}$$

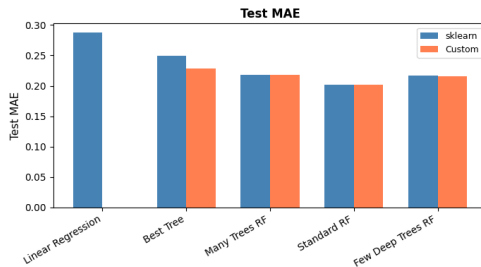
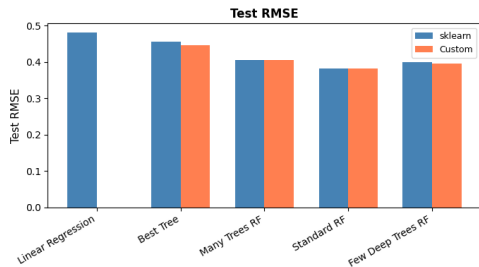
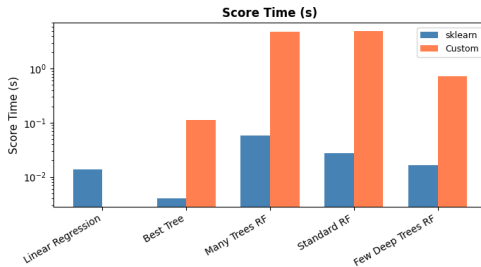
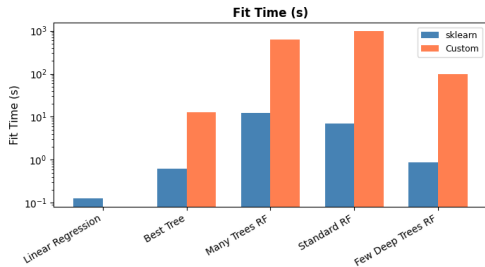
$$\text{MAE}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- 5-fold Cross Validation was performed to evaluate the performance of all models, using the training data of the splits as input.
- Fit- and Scoring times were measured to compare the runtimes of different implementations.

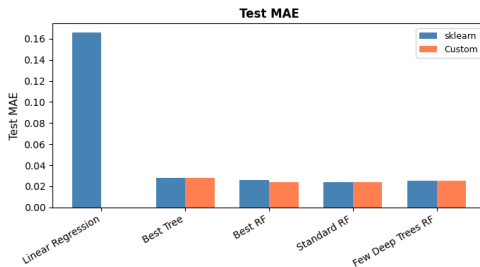
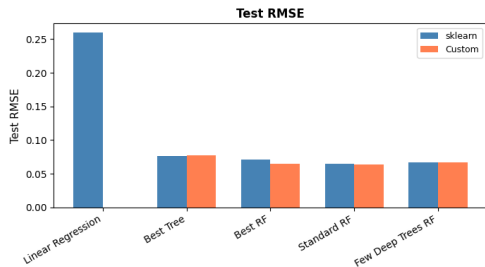
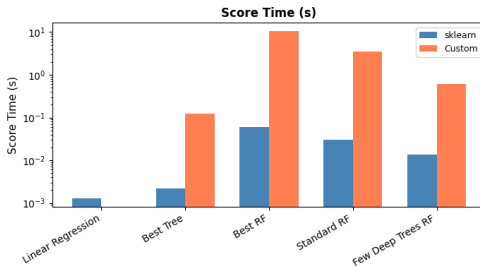
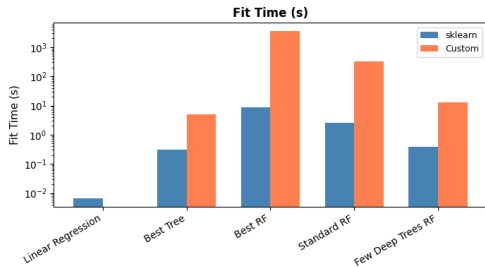
Sklearn vs Custom - Pollen



Sklearn vs Custom - Spotify



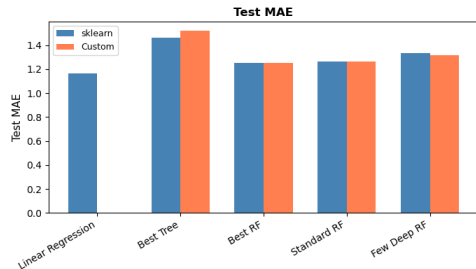
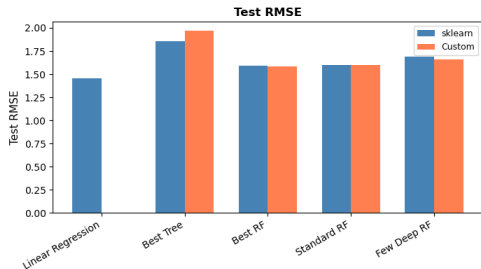
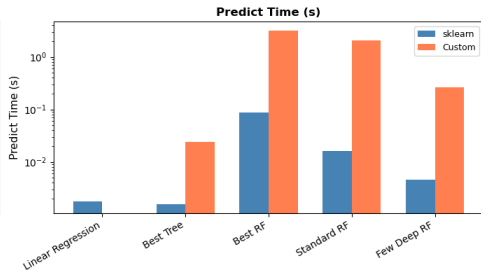
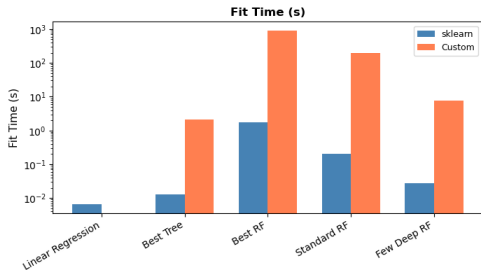
Sklearn vs Custom - Taiwan AQI



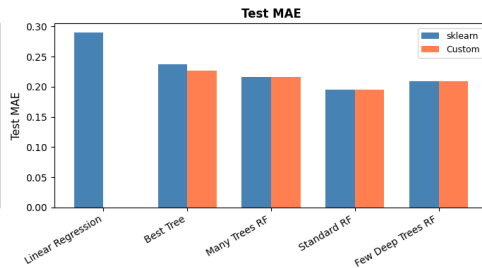
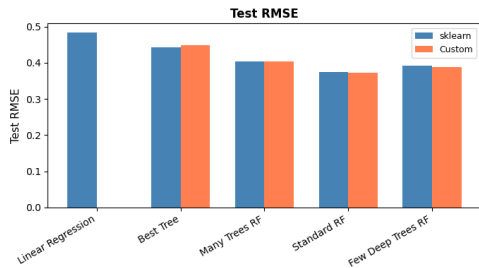
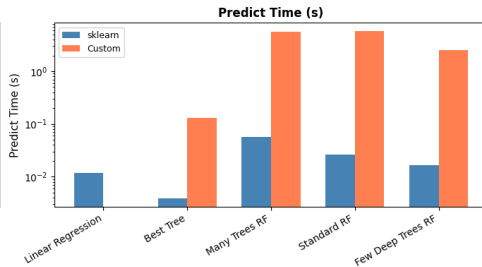
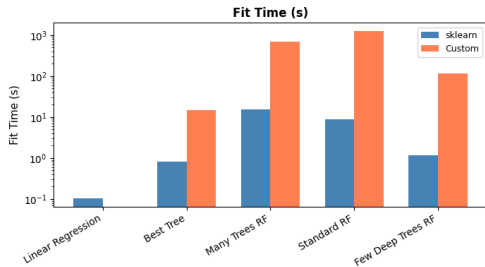
Holdout Validation

- Additionally to Cross Validation, we tested the algorithms on the holdout test dataset (20% of the clean datasets).
- The Fit times and Predict times were again measured to get a feeling for singly called algorithms, where initialization overhead might play a role.

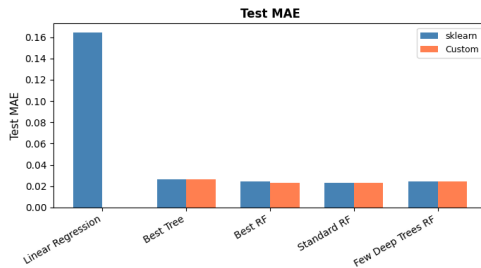
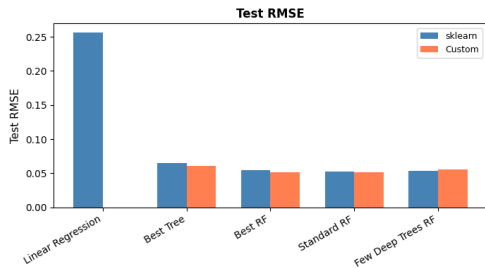
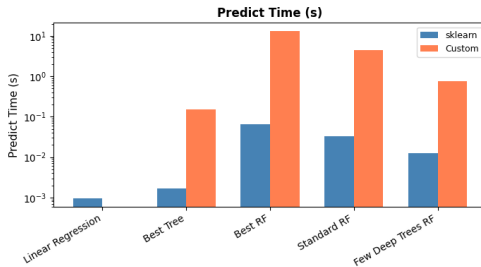
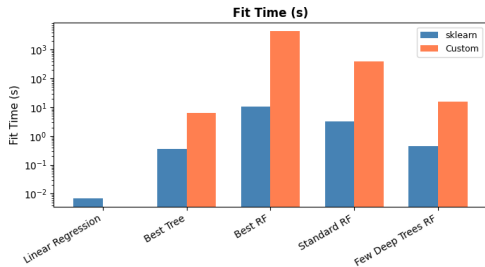
Sklearn vs Custom - Pollen



Sklearn vs Custom - Spotify



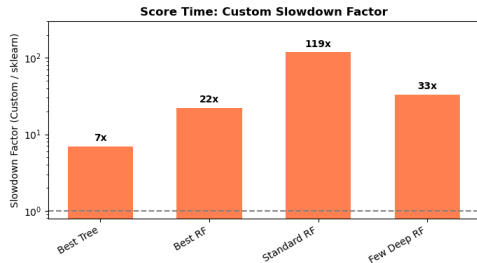
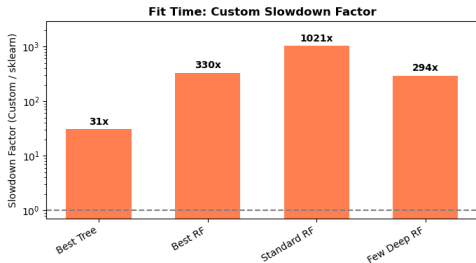
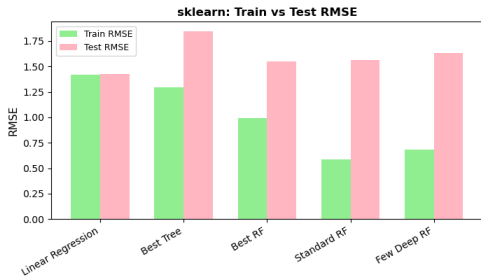
Sklearn vs Custom - Taiwan AQI



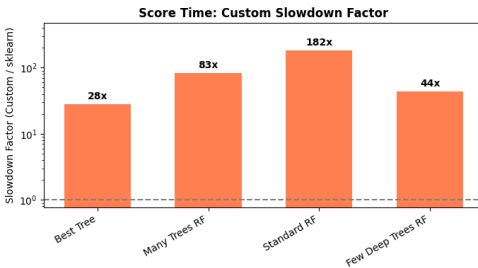
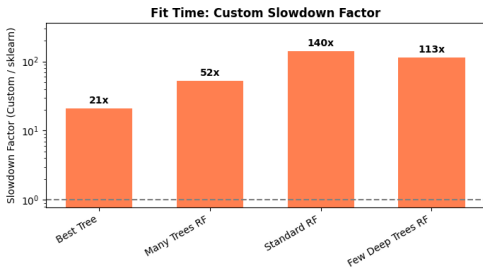
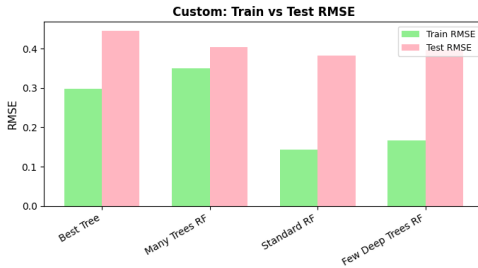
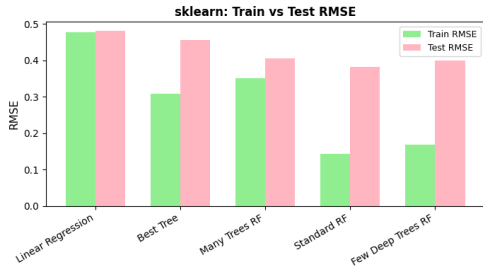
Overfitting & Slowdown

- We compared the Train/Test RMSEs and MAEs to each other to get a sense of overfitting for all models, custom and Sklearn.
- Additionally, we show the actual slowdown using our custom implementations.

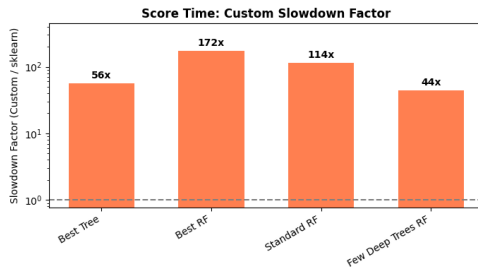
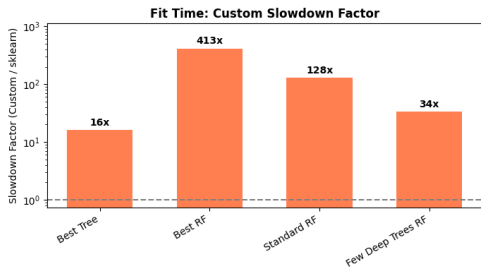
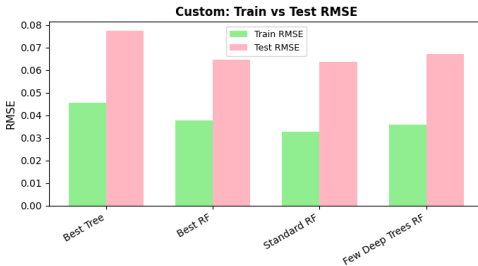
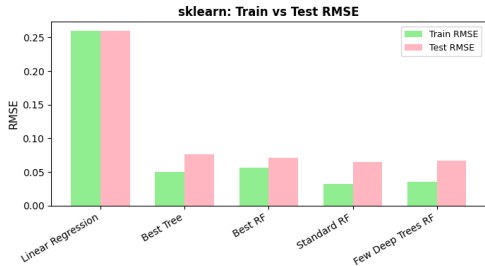
Sklearn vs Custom - Pollen



Sklearn vs Custom - Spotify



Sklearn vs Custom - Taiwan AQI



Optimization Strategy: Randomized Search with Cross-Validation.

Decision Tree (Best Config)

- max_depth: **22**
- min_samples_split: **8**
- min_samples_leaf: **9**
- criterion: *'squared_error'*

Random Forest (Best Config)

- n_estimators: **1347**
- max_depth: **23**
- min_samples_split: **10**
- min_samples_leaf: **1**
- max_features: None (Uses all features)
- bootstrap: True

Used Hyperparameters - Spotify

Optimization Strategy: Randomized Search with Cross-Validation.

Decision Tree (Best Config)

- max_depth: **20**
- min_samples_split: **19**
- min_samples_leaf: **4**
- criterion: *'squared_error'*

Random Forest (Standard Config)

- n_estimators: **100**
- max_depth: **None**
- min_samples_split: **2**
- min_samples_leaf: **1**
- max_features: **None** (Uses all features)
- bootstrap: **True**
- Best Config not used (Runtime: 1000 Trees, 50 depth)

Optimization Strategy: Randomized Search with Cross-Validation.

Decision Tree (Best Config)

- max_depth: **22**
- min_samples_split: **16**
- min_samples_leaf: **2**
- criterion: *'squared_error'*

Random Forest (Best Config)

- n_estimators: **430**
- max_depth: **34**
- min_samples_split: **11**
- min_samples_leaf: **5**
- max_features: None (Uses all features)
- bootstrap: True

- Semi successfully implemented Regression Tree and Random Forest from scratch.
- **Efficiency:** Vectorized NumPy operations and parallelization significantly improved training time compared to naive loops. But performance is far from comparable to C/C++ implementations. Using openMP/MPI or similar is just much faster, but out of scope for this lecture.
- **Accuracy:** Achieved comparable accuracy to Scikit-Learn on all datasets.
- **Learning:** Hard to make a better implementation than Scikit-Learn and not worth trying in languages like python.

References



Joaquin Vanschoren.

Pollen.

<https://www.openml.org/search?type=data&sort=runs&id=529&status=active>.

Accessed: 2025-12-14; dataset for regression tasks with pollen grains features.



Rudra Prasad Bhuyan.

Spotify dataset for ml practice.

[https://www.kaggle.com/datasets/rudraprasadbhuyan/](https://www.kaggle.com/datasets/rudraprasadbhuyan/spotify-dataset-for-ml-practice)

[spotify-dataset-for-ml-practice](https://www.kaggle.com/datasets/rudraprasadbhuyan/spotify-dataset-for-ml-practice), 2025.

Accessed: 2025-12-14; dataset for regression and classification tasks with Spotify audio features.



Taiwan air quality index data 2016-2024.

[https:](https://www.kaggle.com/datasets/taweilo/taiwan-air-quality-data-20162024)

[//www.kaggle.com/datasets/taweilo/taiwan-air-quality-data-20162024](https://www.kaggle.com/datasets/taweilo/taiwan-air-quality-data-20162024).

Accessed: 2025-12-14; dataset for regression and classification tasks with Taiwan AQI.